GateKeep — Product Requirements Document

2026-09-17 · @Someone

Overview

GateKeep is a governed multi-agent pipeline that turns a feature request into tested code for small Python web apps, with a human approval gate between every phase. AI agents write requirements, design, code and tests; humans approve each step; every approval is stored in a tamper-evident audit log.

This is a solo portfolio project, scoped to be built in 8–10 weeks part-time. It is not a commercial product.

Tagline: Agents do the work. Humans sign off. Every decision is on record.

Problem

AI coding tools write code quickly, but teams struggle to trust that code. Three gaps cause most of the pain:

- **No agreement before coding.** The AI starts building before anyone confirms what should be built, so wrong assumptions surface late.
- **No proof of approval.** Nobody can show who approved the requirements, the design or the AI-written code.
- **Broken first drafts.** Generated code often fails tests, and a human re-prompts again and again.

Example: "Add password reset to our Flask app"

Today: A developer asks an AI assistant for the feature. The code emails a reset link but never expires it. Tests fail and the developer re-prompts several times. The product manager later says she only wanted email, not SMS. Nobody wrote down what was agreed, so work is redone. If an auditor asks who approved the auth code, there is no answer.

With GateKeep:

1. The user submits the brief.
2. The Requirements agent writes testable requirements, such as "reset link expires in 30 minutes" and "link works once."

3. **Gate 1:** The pipeline pauses. The PM edits and approves. Her name, time and decision are logged.
4. The Design agent proposes endpoints, a `reset_tokens` table and the flow.
5. **Gate 2:** A senior developer approves the design. Logged.
6. The Code agent writes code and tests, runs them in Docker, reads two failures and fixes them.
7. **Gate 3:** A developer reviews the diff and green test run, then approves. Logged.
8. Anyone can verify the audit chain end to end.

If the server restarts while waiting at Gate 1 over a weekend, the pipeline resumes exactly where it stopped.

Goals and scope

The goal is one pipeline that works end to end on small tasks, with measured results, rather than many half-built features.

Goals

1. Run brief → requirements → design → code → passing tests with three human gates.
2. Pause at gates for days and resume correctly after a server restart.
3. Verify generated code by running tests in an isolated Docker container, with automatic retries.
4. Record every gate decision in a SHA-256 hash-chained audit log that detects edits.
5. Publish honest evaluation numbers: pass rate, cost and retries on 20–30 tasks.

Scope

Area	In scope (v1)	Stretch	Out of scope
Target code	Small Python FastAPI or Flask apps, one service	Existing small repo (brownfield)	Any language, large enterprise codebases
Pipeline	4 agents, 3 gates, reject-with-feedback loop	Contradiction check before Gate 1	Fully autonomous delivery
Execution	Docker container per run, no network, CPU/time limits	Semgrep scan on generated code	gVisor, Kubernetes

Area	In scope (v1)	Stretch	Out of scope
Governance	Hash-chained log, verify endpoint, role check on approvals, lock requirements after Gate 1	STRIDE threat notes from design	Compliance certification claims
Integrations	None required; export result as a zip or git branch	Open a GitHub PR via API	Jira, Slack, GitLab, Azure DevOps
Users	Single team, seeded users with roles	Simple login with JWT	Multi-tenant orgs, SSO, billing
UI	3 screens	Live event stream	16 screens, voice or document intake

Users and pipeline flow

Three roles use GateKeep, and only the right role can approve each gate.

Role	Example person	Can do
Requester	Developer or PM	Create a pipeline from a brief, view progress
Planner	Product manager	Approve or reject Gate 1 (requirements)
Engineer	Senior developer	Approve or reject Gate 2 (design) and Gate 3 (code)

```
flowchart LR
  B[Brief] --> R[Requirements agent]
  R --> G1{Gate 1<br/>Planner}
  G1 -- reject + feedback --> R
  G1 -- approve --> D[Design agent]
  D --> G2{Gate 2<br/>Engineer}
  G2 -- reject + feedback --> D
  G2 -- approve --> C[Code + test agent]
  C --> S[Docker test run]
  S -- fail, retries left --> C
  S -- pass or retries used --> G3{Gate 3<br/>Engineer}
  G3 -- reject + feedback --> C
  G3 -- approve --> O[Output branch]
```

Every gate decision writes one entry to the audit log. If tests still fail after 3 attempts, the run goes to Gate 3 marked as failing, with all attempts attached.

Pipeline states

`running` → `waiting_gate_1` → `running` → `waiting_gate_2` → `running` → `waiting_gate_3` → `completed`. A run can also be `failed` (unrecoverable error) or `cancelled`.

Functional requirements

Each requirement has an ID and a test you can write for it. Priority P0 is required for v1; P1 is stretch.

Agents

ID	Requirement	Acceptance criteria	Priority
AG-1	Requirements agent turns a brief into 3–10 user stories	Output validates against a Pydantic schema; each story has Given/When/Then criteria	P0
AG-2	Design agent proposes endpoints, data model and file plan	Output validates; every story maps to at least one endpoint	P0
AG-3	Code agent writes app code and pytest tests from the approved design	Files are written to the run workspace; Python files parse without syntax errors	P0
AG-4	Agents accept reviewer feedback on rejection	A rejected phase reruns with the feedback in its prompt, and a new version is stored	P0
AG-5	Contradiction check flags conflicting requirements before Gate 1	Seeded conflicting brief produces at least one flag	P1

Durable gates

ID	Requirement	Acceptance criteria	Priority
GT-1	Pipeline pauses at each gate using LangGraph `interrupt()`	Run status becomes `waiting_gate_N`; no agent runs until a decision	P0

ID	Requirement	Acceptance criteria	Priority
GT-2	State is checkpointed in Postgres	Kill the server at a gate, restart, approve: the run continues from the next phase	P0
GT-3	Approve moves forward; reject returns to the phase with feedback	Both paths covered by integration tests	P0
GT-4	Only the correct role can decide a gate	Wrong role gets HTTP 403 from the API, not just a hidden button	P0
GT-5	Requirements are locked after Gate 1 approval	Edit request returns HTTP 423 Locked	P1

Sandbox and self-correction

ID	Requirement	Acceptance criteria	Priority
SB-1	Tests run in a fresh Docker container per attempt	Container has no network, 1 CPU, 1 GB RAM, 120 s timeout, and is removed after	P0
SB-2	Failed runs feed errors back to the code agent	Attempt 2 prompt contains the pytest failure output (trimmed to the last 200 lines)	P0
SB-3	Maximum 3 attempts per Gate 3 cycle	After 3 failures, run goes to Gate 3 marked failing, with all attempts attached	P0
SB-4	Static security scan on generated code	Semgrep results shown at Gate 3	P1

Audit log

ID	Requirement	Acceptance criteria	Priority
AU-1	Each gate decision is stored as an append-only entry	Entry holds run, gate, decision, user, time, feedback and a hash of the approved artifact	P0

ID	Requirement	Acceptance criteria	Priority
AU-2	Entries are hash-chained	`hash = sha256(prev_hash + canonical_json(entry))`; first entry uses a fixed genesis value	P0
AU-3	Verify endpoint checks the chain	Manually editing any row in the database makes verification fail and names the first broken entry	P0
AU-4	App database user cannot update or delete log rows	Postgres grants allow only INSERT and SELECT for the app role	P1

Cost and UI

ID	Requirement	Acceptance criteria	Priority
CO-1	Token usage and cost recorded per agent call	Run detail shows total cost and cost per phase	P0
UI-1	Pipeline list screen	Shows runs with status, current gate and cost	P0
UI-2	Run detail and gate screen	Shows agent output for the current phase, diff and test results at Gate 3, and Approve/Reject with a feedback box	P0
UI-3	Audit timeline screen	Lists entries and shows the result of chain verification	P0
INT-1	Open a GitHub PR after Gate 3	PR created on a test repo with the generated branch	P1

Architecture

A single FastAPI service runs the LangGraph pipeline, stores state in Postgres and launches Docker containers for tests.

```
flowchart LR
  UI[React UI] --> API[FastAPI]
  API --> LG[LangGraph pipeline]
  LG --> LLM[LLM API]
  LG --> DK[Docker test runner]
  LG --> PG[(Postgres<br/>checkpoints, runs, audit)]
  API --> PG
```

The UI polls run status every few seconds in v1. A WebSocket event stream is a stretch goal.

Tech stack

Layer	Choice	Why
Orchestration	LangGraph with `interrupt()` and `AsyncPostgresSaver`	Native pause/resume and durable checkpoints
API	FastAPI, Pydantic	Async, schema validation, role checks as dependencies
Database	PostgreSQL 16	Checkpoints, run data and audit log in one place
LLM	One provider API, a stronger model for code and a cheaper one for requirements	Simple routing that is easy to explain
Sandbox	Docker SDK for Python	Isolated, disposable test runs
Frontend	React + Vite + TypeScript	Standard and fast to build
Testing	pytest, a mocked LLM for unit tests	Deterministic tests without API cost
Local setup	Docker Compose	One command to run everything

Data model

Table	Key columns
`users`	id, name, role (requester, planner, engineer)
`pipeline_runs`	id, brief, status, current_gate, total_cost_usd, created_by, created_at

Table	Key columns
`artifacts`	id, run_id, phase, version, content_json, content_hash, locked
`test_attempts`	id, run_id, attempt_no, passed, failed_count, log_tail, duration_s
`llm_calls`	id, run_id, agent, model, input_tokens, output_tokens, cost_usd
`audit_log`	seq, run_id, gate, decision, user_id, feedback, artifact_hash, created_at, prev_hash, hash

LangGraph's own checkpoint tables are created by `AsyncPostgresSaver.setup()`.

API

Method	Path	Purpose	Role
POST	`/runs`	Start a pipeline from a brief	Requester
GET	`/runs`	List runs	Any
GET	`/runs/{id}`	Run status, current phase output, cost	Any
POST	`/runs/{id}/gates/{n}/decision`	Approve or reject with feedback	Planner (gate 1), Engineer (gates 2–3)
PATCH	`/runs/{id}/requirements`	Edit requirements before Gate 1	Planner; 423 after approval
GET	`/runs/{id}/attempts`	Test attempts and logs	Any
GET	`/audit`	Audit entries, filter by run	Any
GET	`/audit/verify`	Verify the hash chain	Any

Security notes

- Generated code only runs inside containers with no network access and resource limits.

- The audit log is tamper-evident, not tamper-proof. A database superuser can rewrite the whole chain. A stretch fix is to publish the latest hash somewhere external, such as a daily git commit.
- API keys live in environment variables and are never passed into containers.

Evaluation and success metrics

The evaluation proves the pipeline works and shows where it fails. Results go in the README.

Benchmark set

- 25 small feature tasks on a starter Flask or FastAPI app, for example password reset, pagination, rate limiting and CSV export.
- Each task has a brief plus 5–15 **hidden** tests written by you. The agent never sees them.
- Split: 10 easy, 10 medium, 5 hard.
- Gates are auto-approved in eval mode, so results measure the agents rather than the reviewer. Auto-approval is only possible with an eval flag and is written to the log.

Metrics

Metric	How it is measured	Target
Task pass rate	Tasks where all hidden tests pass	Report honestly; aim for 60%+
First-attempt pass rate	Hidden tests pass without any retry	Report, to show the value of retries
Average attempts	Mean code attempts per task	2 or fewer
Cost per task	Sum of `llm_calls.cost_usd`	Under $1.00
Time per task	Brief to Gate 3, gates excluded	Under 10 minutes
Restart recovery	Kill the server at each gate in 5 runs	5 of 5 resume correctly
Tamper detection	Edit, delete or reorder log rows in 10 trials	10 of 10 detected

Failure analysis

Label every failed task with one cause: misunderstood requirement, wrong design, code bug not fixed, test environment problem, or timeout. Report the counts. This table is often more impressive in interviews than the pass rate.

Ablation

Run the same tasks with retries turned off, and with the design phase skipped. Compare pass rate and cost against the full pipeline.

How it differs from PR review

Gate 3 is essentially a PR review; Gates 1 and 2 and the linked audit trail are the additions.

Question	GateKeep gate	Covered by a normal PR review?
Are we building the right thing?	Gate 1, requirements	No
Is this the right approach?	Gate 2, design	No
Is the code correct and tested?	Gate 3, code	Yes

GitHub's coding agent already writes code and opens PRs for human review, so GateKeep does not claim to be unique. Its angle is approvals earlier in the process, tests run before a human looks, and one chain linking all decisions.

Limitations and risks

Risk or limitation	Impact	Mitigation
Code agent fails on harder tasks	Low pass rate	Keep tasks small; report failures honestly with causes
LLM costs during eval runs	Budget overrun	Cheap model for early phases; cache outputs; run full eval only a few times
Hash chain can be rewritten by a DB superuser	Weaker tamper claim	Call it tamper-evident; optional external hash anchoring
Docker isolation is not a hardened sandbox	Unsafe for untrusted production use	No network, resource limits; state the limit in the README

Risk or limitation	Impact	Mitigation
Only small Python apps supported	Limited realism	State the scope clearly as a design choice
Scope creep	Nothing finished	Follow the out-of-scope list; P1 items only after P0 is done

Claims to avoid

- "SOC2 or EU AI Act compliant"
- "Undeletable" or "tamper-proof" audit log
- "No competitors" or "fully autonomous delivery"

Resume presentation

Use your own results in place of X, Y and Z.

GateKeep: Governed Multi-Agent Software Pipeline · Python, LangGraph, FastAPI, PostgreSQL, Docker, React

- Built a multi-agent pipeline that turns feature requests into tested code through requirements, design and coding agents, with human approval gates that survive server restarts via Postgres checkpointing
- Built a code agent that runs tests in isolated Docker containers and self-corrects from failures, passing X/25 benchmark tasks at an average of $Y and Z attempts per task
- Recorded all approvals in a SHA-256 hash-chained audit log with tamper verification; enforced role-based approvals and requirement locking at the API layer

20-second pitch: AI can write code, but teams can't trust it without agreed requirements, human sign-off and proof of who approved what. I built a pipeline where agents do each phase, humans approve each step, runs survive restarts, code is verified in a sandbox, and every approval is in a tamper-evident log. Then I measured how well it works.

System design concerns, scenario by scenario

Each row below is a concrete failure scenario, why it matters, and the concrete fix to build.

Pipeline orchestration

Scenario	Concern	Fix
Two people approve the same gate at once	Race condition; pipeline could advance twice or corrupt state	Use a Postgres row lock (`SELECT ... FOR UPDATE`) on the run row before applying a gate decision; second request gets 409 Conflict
Server crashes mid-agent-call, not at a gate	LangGraph checkpoints between nodes, not mid-node, so an in-flight LLM call is lost	On restart, detect a run stuck in a non-gate state for over N minutes and re-run that node; make agent nodes idempotent (write output only after the call fully returns)
Two server instances both poll for runs to resume	Both try to resume the same run, causing duplicate LLM calls or double writes	Use Postgres advisory locks keyed by run ID before resuming, or run a single-worker resume process for v1
A gate is approved, but the reviewer's browser tab was showing stale data	Reviewer approves requirements they didn't actually see, e.g. another reviewer edited them first	Send the content hash the reviewer saw with the approval request; reject if it doesn't match the current artifact hash
Rejection loop never converges	Agent keeps producing output the reviewer rejects, wasting cost	Cap rejections per phase, e.g. 3; after that, pause and require a human to edit the artifact directly instead of re-prompting the agent

Sandbox and code execution

Scenario	Concern	Fix
Generated code tries to read files outside its folder	Container has filesystem access to more than the run's own files	Mount only the run's workspace directory into the container, read-write; everything else read-only or absent
Generated test code calls a shell command or opens a socket	Malicious or reckless code escapes the intended scope	No network access, drop Linux capabilities, run as a non-root user inside the container
A test hangs forever, an infinite loop	Container never exits, run stalls, resources leak	Hard timeout on the container, e.g. 120 seconds, force stop, treated as a failed attempt
Two runs' containers collide on the same port or volume name	Flaky failures that look like code bugs but aren't	Namespace container names, ports and volumes by run ID and attempt number
Docker daemon itself is unavailable or overloaded	Whole pipeline silently stalls at the code phase	Health-check the Docker daemon before starting a run; surface a clear sandbox-unavailable error rather than hanging
Disk fills up from accumulated containers and images	Host runs out of space after many eval runs	Auto-remove containers after each attempt; prune images on a schedule

Audit log

Scenario	Concern	Fix
Someone edits a row directly in Postgres	Breaks the hash chain, but only if someone actually runs verify	Run the verify check automatically on a schedule and alert on failure, not just on demand

Scenario	Concern	Fix
Two audit entries get the same previous-hash, a chain fork	Concurrent writes could create two branches instead of one line	Make the sequence column serial, and compute each hash inside the same transaction that inserts the row, so ordering is enforced by the database, not the application
App's database credentials are compromised	Attacker with app-level access could still insert fake entries with valid-looking hashes	Hashing prevents undetected retroactive edits, but not fake forward entries from a compromised app; document this limitation rather than overclaiming
Genesis hash is hardcoded and public	Someone forks the log and claims a fake chain is legitimate	Not a real risk for a portfolio project; worth one sentence in the README explaining why it's out of scope

Multi-agent and LLM concerns

Scenario	Concern	Fix
LLM returns malformed JSON for a structured output	Pydantic validation fails, agent step crashes	Retry with the validation error appended to the prompt, up to 2 times, before surfacing a hard failure to the human
LLM output is non-deterministic across retries	Two identical runs in the eval produce different pass or fail results	Set temperature to zero for structured extraction agents; accept and report variance for the code agent, since that's realistic
Prompt injection from the user's brief	A malicious brief tries to make the design agent ignore instructions, e.g. asking it to mark all gates auto-approved	Keep the brief as untrusted input in the prompt; never let agent output alter gate logic or role checks, which live only in application code

Scenario	Concern	Fix
Cost runaway during eval runs	25 tasks times 3 retries times 4 agents adds up fast if unmonitored	Set a hard per-run cost ceiling; abort and mark failed if exceeded; log cost per call from the first run, not after the fact
Context window overflow on a long-running brownfield repo, a stretch scenario	Design or code agent silently truncates important context	Cap input size, chunk and summarize larger inputs, and log when truncation happens rather than let it fail silently

Data and API

Scenario	Concern	Fix
A planner tries to approve an engineer-only gate	Role check needs to be enforced somewhere that can't be bypassed	Check role in a FastAPI dependency on the route, not in the frontend; return 403, and log the attempt
Requirements are edited after Gate 1 approval	The whole point of locking is defeated if this succeeds	Check a locked flag on the artifact row inside the same transaction as the edit; return 423
Run ID guessed or enumerated by another user	Information disclosure between runs if there's ever more than one team	Not a v1 concern with a single seeded team, but note it as a gap if multi-user auth is added later
Postgres connection pool exhausted under load	API starts timing out under concurrent runs	Set explicit pool size limits and time out slow queries; not likely to matter at demo scale, but worth one line in the README

What to actually build vs. document

You don't need to code every fix above. For a resume project, the strongest move is to implement the fixes with concrete, cheap mechanisms, such as row locks, hashes computed in a transaction, container flags, and role checks in dependencies, and to write

down the ones you're consciously deferring, such as compromised credentials, multi-tenant isolation, and context overflow, in a Known Limitations list. Interviewers care more that you can name the failure modes than that you've closed every one of them.